

# 23

## *Universal Types*

In the previous chapter, we studied the simple form of *let-polymorphism* found in ML. In this chapter, we consider a more general form of polymorphism in the setting of a powerful calculus known as *System F*.

### 23.1 Motivation

As we remarked in §22.7, we can write an infinite number of “doubling” functions in the simply typed lambda-calculus...

```
doubleNat = λf:Nat→Nat. λx:Nat. f (f x);  
doubleRcd = λf:{1:Bool}→{1:Bool}. λx:{1:Bool}. f (f x);  
doubleFun = λf:(Nat→Nat)→(Nat→Nat). λx:Nat→Nat. f (f x);
```

Each of these functions is applicable to a different type of argument, but all share precisely the same behavior (indeed, they share precisely the same program text, aside from the typing annotations). If we want apply the doubling operation to different types of arguments within the same program, we will need to write out separate definitions of `doubleT` for each `T`. This kind of cut-and-paste programming violates a basic dictum of software engineering:

ABSTRACTION PRINCIPLE: Each significant piece of functionality in a program should be implemented in just one place in the source code. Where similar functions are carried out by distinct pieces of code, it is generally beneficial to combine them into one by *abstracting out* the varying parts.

---

The system studied in most of this chapter is pure System F (Figure 23-1); the examples in §23.4 use various extensions with previously studied features. The associated OCaml implementation is `fullpoly`. (The examples involving pairs and lists require the `fullomega` checker.)

Here, the varying parts are the types! What we need, then, are facilities for abstracting out a type from a term and later instantiating this abstract term with concrete type annotations.

## 23.2 Varieties of Polymorphism

Type systems that allow a single piece of code to be used with multiple types are collectively known as *polymorphic* systems (*poly* = many, *morph* = form). Several varieties of polymorphism can be found in modern languages (this classification comes from Strachey, 1967, and Cardelli and Wegner, 1985).

*Parametric polymorphism*, the topic of this chapter, allows a single piece of code to be typed “generically,” using variables in place of actual types, and then instantiated with particular types as needed. Parametric definitions are *uniform*: all of their instances behave the same.

The most powerful form of parametric polymorphism is the *impredicative* or *first-class polymorphism* developed in this chapter. More common in practice is the form known as *ML-style* or *let-polymorphism*, which restricts polymorphism to top-level *let*-bindings, disallowing functions that take polymorphic values as arguments, and obtains in return a convenient and natural form of automatic *type reconstruction* (Chapter 22). First-class parametric polymorphism is also becoming popular in programming languages, and forms the technical foundation for the powerful module systems of languages like ML (see Harper and Stone, 2000).

*Ad-hoc polymorphism*, by contrast, allows a polymorphic value to exhibit different behaviors when “viewed” at different types. The most common example of ad-hoc polymorphism is *overloading*, which associates a single function symbol with many implementations; the compiler (or the run-time system, depending on whether overloading resolution is *static* or *dynamic*) chooses an appropriate implementation for each application of the function, based on the types of the arguments.

A generalization of function overloading forms the basis for *multi-method dispatch* in languages such as CLOS (Bobrow et al., 1988; Kiczales et al., 1991) and Cecil (Chambers, 1992; Chambers and Leavens, 1994). This mechanism has been formalized in the  $\lambda$ -& calculus of Castagna, Ghelli, and Longo (1995; cf. Castagna, 1997).

A more powerful form of ad-hoc polymorphism known as *intensional polymorphism* (Harper and Morrisett, 1995; Cray, Weirich, and Morrisett, 1998) permits restricted computation over types at run time. Intensional polymorphism is an enabling technology for a variety of advanced implemen-

tation techniques for polymorphic languages, including tag-free garbage collection, “unboxed” function arguments, polymorphic marshaling, and space-efficient “flattened” data structures.

Yet more powerful forms of ad-hoc polymorphism can be built from a *typecase* primitive, which permits arbitrary pattern-matching on type information at run time (Abadi, Cardelli, Pierce, and Rémy, 1995; Abadi, Cardelli, Pierce, and Plotkin, 1991b; Henglein, 1994; Leroy and Mauny, 1991; Thatte, 1990). Language features such as Java’s `instanceof` test can be viewed as restricted forms of *typecase*.

The *subtype polymorphism* of Chapter 15 gives a single term many types using the rule of subsumption, allowing us to selectively “forget” information about the term’s behavior.

These categories are not exclusive: different forms of polymorphism can be mixed in the same language. For example, Standard ML offers both parametric polymorphism and simple overloading of built-in arithmetic operations, but not subtyping, while Java includes subtyping, overloading, and simple ad-hoc polymorphism (`instanceof`), but not (at the time of this writing) parametric polymorphism. There are several proposals for adding parametric polymorphism to Java; the best known of these is GJ (Bracha, Odersky, Stoutamire, and Wadler, 1998).

The unqualified term “polymorphism” causes a certain amount of confusion between programming language communities. Among functional programmers (i.e., those who use or design languages like ML, Haskell, etc.), it almost always refers to parametric polymorphism. Among object-oriented programmers, on the other hand, it almost always means subtype polymorphism, while the term *genericity* (or *generics*) is used for parametric polymorphism.

### 23.3 System F

The system we will be studying in this chapter, commonly called *System F*, was first discovered by Jean-Yves Girard (1972), in the context of proof theory in logic. A little later, a type system with essentially the same power was developed, independently, by a computer scientist, John Reynolds (1974), who called it the *polymorphic lambda-calculus*. This system has been used extensively as a research vehicle for foundational work on polymorphism and as the basis for numerous programming language designs. It is also sometimes called the *second-order lambda-calculus*, because it corresponds, via the *Curry-Howard correspondence*, to second-order intuitionistic logic, which allows quantification not only over individuals [terms], but also over predicates [types].

The definition of System F is a straightforward extension of  $\lambda_-$ , the simply typed lambda-calculus. In  $\lambda_-$ , lambda-abstraction is used to abstract terms out of terms, and application is used to supply values for the abstracted parts. Since we want here a mechanism for abstracting *types* out of terms and filling them in later, we introduce a new form of abstraction, written  $\lambda X. t$ , whose parameter is a type, and a new form of application,  $t [T]$ , in which the argument is a type expression. We call our new abstractions *type abstractions* and the new application construct *type application* or *instantiation*.

When, during evaluation, a type abstraction meets a type application, the pair forms a redex, just as in  $\lambda_-$ . We add a reduction rule

$$(\lambda X. t_{12}) [T_2] \rightarrow [X \mapsto T_2] t_{12} \quad (\text{E-TAPPTABS})$$

analogous to the ordinary reduction rule for abstractions and applications.

$$(\lambda x : T_{11}. t_{12}) v_2 \rightarrow [x \mapsto v_2] t_{12} \quad (\text{E-APPABS})$$

For example, when the *polymorphic identity function*

$$\text{id} = \lambda X. \lambda x : X. x;$$

is applied to  $\text{Nat}$  by writing  $\text{id} [\text{Nat}]$ , the result is  $[X \mapsto \text{Nat}] (\lambda x : X. x)$ , i.e.,  $\lambda x : \text{Nat}. x$ , the identity function on natural numbers.

Finally, we need to specify the *type* of a polymorphic abstraction. We use types like  $\text{Nat} \rightarrow \text{Nat}$  for classifying ordinary functions like  $\lambda x : \text{Nat}. x$ ; we now need a different form of “arrow type” whose domain is a type, for classifying polymorphic functions like  $\text{id}$ . Notice that, for each argument  $T$  to which it is applied,  $\text{id}$  yields a function of type  $T \rightarrow T$ ; that is, the type of the result of  $\text{id}$  depends on the actual type that we pass it as argument. To capture this dependency, we write the type of  $\text{id}$  as  $\forall X. X \rightarrow X$ . The typing rules for polymorphic abstraction and application are analogous to the rules for term-level abstraction and application.

$$\frac{\Gamma, X \vdash t_2 : T_2}{\Gamma \vdash \lambda X. t_2 : \forall X. T_2} \quad (\text{T-TABS})$$

$$\frac{\Gamma \vdash t_1 : \forall X. T_{12}}{\Gamma \vdash t_1 [T_2] : [X \mapsto T_2] T_{12}} \quad (\text{T-TAPP})$$

Note that we include the type variable  $X$  in the context used by the subderivation for  $t$ . We continue the convention (5.3.4) that the names of (term or type) variables should be chosen so as to be different from all the names already bound by  $\Gamma$ , and that lambda-bound type variables may be renamed at will in order to satisfy this condition. (In some presentations of System F, this freshness condition is given as an explicit side condition on the T-TABS rule, instead of being built into the rules about how contexts are constructed, as we are doing here.) For the moment, the only role of type variables in contexts

| $\rightarrow \forall$ |                        | Based on $\lambda_{\rightarrow}$ (9-1)                                                                                    |                       |
|-----------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------|
| Syntax                |                        | Evaluation                                                                                                                | $t \rightarrow t'$    |
| $t ::=$               |                        |                                                                                                                           |                       |
| $x$                   | terms:<br>variable     | $\frac{t_1 \rightarrow t'_1}{t_1 \ t_2 \rightarrow t'_1 \ t_2}$                                                           | (E-APP1)              |
| $\lambda x:T. t$      | abstraction            | $\frac{t_2 \rightarrow t'_2}{v_1 \ t_2 \rightarrow v_1 \ t'_2}$                                                           | (E-APP2)              |
| $t \ t$               | application            | $(\lambda x:T_{11}. t_{12}) \ v_2 \rightarrow [x \mapsto v_2] t_{12}$                                                     | (E-APPABS)            |
| $\lambda X. t$        | type abstraction       | $\frac{t_1 \rightarrow t'_1}{t_1 \ [T_2] \rightarrow t'_1 \ [T_2]}$                                                       | (E-TAPP)              |
| $t \ [T]$             | type application       | $(\lambda X. t_{12}) \ [T_2] \rightarrow [X \mapsto T_2] t_{12}$                                                          | (E-TAPPTABS)          |
| $v ::=$               | values:                |                                                                                                                           |                       |
| $\lambda x:T. t$      | abstraction value      |                                                                                                                           |                       |
| $\lambda X. t$        | type abstraction value |                                                                                                                           |                       |
| $T ::=$               | types:                 |                                                                                                                           |                       |
| $X$                   | type variable          | Typing                                                                                                                    | $\Gamma \vdash t : T$ |
| $T \rightarrow T$     | type of functions      | $\frac{x:T \in \Gamma}{\Gamma \vdash x : T}$                                                                              | (T-VAR)               |
| $\forall X. T$        | universal type         | $\frac{\Gamma, x:T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x:T_1. t_2 : T_1 \rightarrow T_2}$                           | (T-ABS)               |
| $\Gamma ::=$          | contexts:              | $\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 \ t_2 : T_{12}}$ | (T-APP)               |
| $\emptyset$           | empty context          | $\frac{\Gamma, X \vdash t_2 : T_2}{\Gamma \vdash \lambda X. t_2 : \forall X. T_2}$                                        | (T-TABS)              |
| $\Gamma, x:T$         | term variable binding  | $\frac{\Gamma \vdash t_1 : \forall X. T_{12}}{\Gamma \vdash t_1 \ [T_2] : [X \mapsto T_2] T_{12}}$                        | (T-TAPP)              |
| $\Gamma, X$           | type variable binding  |                                                                                                                           |                       |

Figure 23-1: Polymorphic lambda-calculus (System F)

is to keep track of scopes and make sure that the same type variable is not added twice to a context. In later chapters, we will annotate type variables with information of various kinds, such as *bounds* (Chapter 26) and *kinds* (Chapter 29).

Figure 23-1 shows the complete definition of the polymorphic lambda-calculus, with differences from  $\lambda_{\rightarrow}$  highlighted. As usual, this summary defines just the pure calculus, omitting other type constructors such as records,

base types such as `Nat` and `Bool`, and term-language extensions such as `let` and `fix`. These extra constructs can be added straightforwardly to the pure system, and we will use them freely in the examples that follow.

## 23.4 Examples

We now develop several examples of programming with polymorphism. To warm up, we start with a few small but increasingly tricky examples, showing some of the expressive power of System F. We then review the basic ideas of “ordinary” polymorphic programming with lists, trees, etc. The last two subsections introduce typed versions of the Church encodings of simple algebraic datatypes like booleans, numbers, and lists that we saw in Chapter 5 for the untyped lambda-calculus. Although these encodings are of small practical importance—it is easier to compile good code for these important features if they are built into high-level programming languages as primitives—they make excellent exercises for understanding both the intricacies and the power of System F. In Chapter 24, we will see some additional applications of polymorphism in the domain of *modular programming* and *abstract data types*.

### Warm-ups

We have seen already how type abstraction and application can be used to define a single polymorphic identity function

```
id = λX. λx:X. x;
```

► `id : ∀X. X → X`

and instantiate it to yield any concrete identity function that may be required:

```
id [Nat];
```

► `<fun> : Nat → Nat`

```
id [Nat] 0;
```

► `0 : Nat`

A more useful example is the polymorphic doubling function:

```
double = λX. λf:X→X. λa:X. f (f a);
```

► `double : ∀X. (X→X) → X → X`

The abstraction on the type  $X$  allows us to obtain doubling functions for specific types by instantiating `double` with different type arguments:

```
doubleNat = double [Nat];
```

► `doubleNat : (Nat → Nat) → Nat → Nat`

```
doubleNatArrowNat = double [Nat → Nat];
```

► `doubleNatArrowNat : ((Nat → Nat) → Nat → Nat) → (Nat → Nat) → Nat → Nat`

Once instantiated with a type argument, `double` can be further applied to an actual function and argument of appropriate types:

```
double [Nat] (λx:Nat. succ(succ(x))) 3;
```

► `7 : Nat`

Here is a slightly trickier example: polymorphic self-application. Recall that, in the simply typed lambda-calculus, there is no way to type the untyped term  $\lambda x. x\ x$  (Exercise 9.3.2). In System F, on the other hand, this term becomes typable if we give  $x$  a polymorphic type and instantiate it appropriately:

```
selfApp = λx:∀X.X→X. x [∀X.X→X] x;
```

► `selfApp : (∀X. X → X) → (∀X. X → X)`

As a (slightly) more useful example of self application, we can apply the polymorphic `double` function to itself, yielding a polymorphic quadrupling function:

```
quadruple = λX. double [X→X] (double [X]);
```

► `quadruple : ∀X. (X → X) → X → X`

23.4.1 EXERCISE [★ →]: Using the typing rules in Figure 23-1, convince yourself that the terms above have the types given. □

### Polymorphic Lists

Most real-world programming with polymorphism is much more pedestrian than the tricky examples above. As an example of straightforward polymorphic programming, suppose our programming language is equipped with a type constructor `List` and term constructors for the usual list manipulation primitives, with the following types.

```

► nil : ∀X. List X
  cons : ∀X. X → List X → List X
  isnil : ∀X. List X → Bool
  head : ∀X. List X → X
  tail : ∀X. List X → List X

```

When we first introduced lists in §11.12, we used “custom” inference rules to allow the operations to be applied to lists with elements of any type. Here, we can give the operations polymorphic types expressing exactly the same constraints—that is, lists no longer need to be “baked into” the core language, but can simply be considered as a library providing several constants with particular polymorphic types. The same holds for the `Ref` type and the primitive operations on reference cells in Chapter 13, and many other common data and control structures.

We can use these primitives to define our own polymorphic operations on lists. For example, here is a polymorphic `map` function that takes a function from `X` to `Y` and a list of `X`s and returns a list of `Y`s.

```

map = λX. λY.
      λf: X→Y.
        (fix (λm: (List X) → (List Y).
              λl: List X.
                if isnil [X] 1
                  then nil [Y]
                  else cons [Y] (f (head [X] 1))
                               (m (tail [X] 1))));
► map : ∀X. ∀Y. (X→Y) → List X → List Y
  1 = cons [Nat] 4 (cons [Nat] 3 (cons [Nat] 2 (nil [Nat])));
► 1 : List Nat
  head [Nat] (map [Nat] [Nat] (λx:Nat. succ x) 1);
► 5 : Nat

```

23.4.2 EXERCISE [★ →]: Convince yourself that `map` really has the type shown. □

23.4.3 EXERCISE [RECOMMENDED, ★★]: Using `map` as a model, write a polymorphic list-reversing function:

```
reverse : ∀X. List X → List X.
```

This exercise is best done on-line. Use the `fullomega` checker and copy the contents of the file `test.f` from the `fullomega` directory to the top of your own input file. (The file contains definitions of the `List` constructor and associated operations that require the powerful abstraction facilities of System  $F_\omega$ , discussed in Chapter 29. You do not need to understand exactly how they work to proceed with the present exercise.) □

## 23.4.4 EXERCISE [★★ →]: Write a simple polymorphic sorting function

$$\text{sort} : \forall X. (X \rightarrow X \rightarrow \text{Bool}) \rightarrow (\text{List } X) \rightarrow \text{List } X$$

where the first argument is a comparison function for elements of type  $X$ .  $\square$

### Church Encodings

We saw in §5.2 that a variety of primitive data values such as booleans, numbers, and lists can be encoded as functions in the pure untyped lambda-calculus. In this section, we show how these *Church encodings* can also be carried out in System F. Readers may want to refer to §5.2 to refresh their intuitions about Church encodings.

These encodings are interesting for two reasons. First, they give our understanding of type abstraction and application a good workout. Second, they demonstrate that System F, like the pure untyped lambda-calculus, is computationally a very rich language, in the sense that the pure system can express a large range of data and control structures. This means that, if we later design a full-scale programming language with System F as its core, we can add these features as primitives (for efficiency, and so that we can equip them with more convenient concrete syntax) without disturbing the fundamental properties of the core language. This will not be true of all interesting high-level language features, of course. For example, adding *references* to System F, as we did for  $\lambda_{\perp}$  in Chapter 13, represents a real change in the fundamental computational nature of the system.

Let us begin with the Church booleans. Recall that, in the untyped lambda-calculus, we represented the boolean constants `true` and `false` by lambda-terms `tru` and `f1s` defined like this:

```
tru = λt. λf. t;
f1s = λt. λf. f;
```

Each of these terms takes two arguments and returns one of them. If we want to assign a common type to `tru` and `f1s`, we had better assume that the two arguments have the same type (since the caller will not know whether it is interacting with `tru` or `f1s`), but this type may be arbitrary (since `tru` and `f1s` do not need to do anything with their arguments except return one of them). This leads us to the following type for `tru` and `f1s`.

$$\text{CBool} = \forall X. X \rightarrow X \rightarrow X;$$

The System-F terms `tru` and `f1s` are obtained by adding appropriate type annotations to the untyped versions above.

```

tru = λX. λt:X. λf:X. t;
► tru : CBool

fls = λX. λt:X. λf:X. f;
► fls : CBool

```

We can write common boolean operations like `not` by constructing a new boolean that uses an existing one to decide which of its arguments to return:

```

not = λb:CBool. λX. λt:X. λf:X. b [X] f t;
► not : CBool → CBool

```

23.4.5 EXERCISE [RECOMMENDED, ★]: Write a term and that takes two arguments of type `CBool` and computes their conjunction. □

We can play a similar game with numbers. The *Church numerals* introduced in §5.2 encode each natural number  $n$  as a function that take two arguments  $s$  and  $z$  and applies  $s$  to  $z$ ,  $n$  times:

```

c0 = λs. λz. z;
c1 = λs. λz. s z;
c2 = λs. λz. s (s z);
c3 = λs. λz. s (s (s z));

```

Clearly, the  $z$  argument should have the same type as the domain of  $s$ , and the result returned by  $s$  should again have the same type. This leads us to the following type for Church numerals in System F:

```

CNat = ∀X. (X→X) → X → X;

```

The elements of this type are obtained by adding appropriate annotations to the untyped Church numerals:

```

c0 = λX. λs:X→X. λz:X. z;
► c0 : CNat

c1 = λX. λs:X→X. λz:X. s z;
► c1 : CNat

c2 = λX. λs:X→X. λz:X. s (s z);
► c2 : CNat

```

A typed successor function on Church numerals can be defined as follows.

```
csucc = λn:CNat. λX. λs:X→X. λz:X. s (n [X] s z);
```

► `csucc : CNat → CNat`

That is, `csucc n` returns an element of `CNat` that, given `s` and `z`, applies `s` to `z`, `n` times (by applying `n` to `s` and `z`), and then once more. Other arithmetic operations can be defined similarly. For example, addition can be defined either in terms of successor,

```
cplus = λm:CNat. λn:CNat. m [CNat] csucc n;
```

► `cplus : CNat → CNat → CNat`

or more directly:

```
cplus = λm:CNat. λn:CNat. λX. λs:X→X. λz:X. m [X] s (n [X] s z);
```

► `cplus : CNat → CNat → CNat`

If our language also includes primitive numbers (Figure 8-2), then we can convert from Church numerals to ordinary ones using the following function:

```
cnat2nat = λm:CNat. m [Nat] (λx:Nat. succ(x)) 0;
```

► `cnat2nat : CNat → Nat`

This allows us to verify that our operations on Church numerals actually compute the desired arithmetic functions:

```
cnat2nat (cplus (csucc c0) (csucc (csucc c0)));
```

► `3 : Nat`

23.4.6 EXERCISE [RECOMMENDED, ★★]: Write a function `iszero` that will return `true` when applied to the Church numeral `c0` and `false` otherwise. □

23.4.7 EXERCISE [★★ →]: Verify that the terms

```
ctimes = λm:CNat. λn:CNat. λX. λs:X→X. n [X] (m [X] s);
```

► `ctimes : CNat → CNat → CNat`

```
cexp = λm:CNat. λn:CNat. λX. n [X→X] (m [X]);
```

► `cexp : CNat → CNat → CNat`

have the indicated types. Give an informal argument that they implement the arithmetic multiplication and exponentiation operators. □

23.4.8 EXERCISE [RECOMMENDED, ★★]: Show that the type

$$\text{PairNat} = \forall X. (\text{CNat} \rightarrow \text{CNat} \rightarrow X) \rightarrow X;$$

can be used to represent pairs of numbers, by writing functions

```
pairNat : CNat → CNat → PairNat;
fstNat  : PairNat → CNat;
sndNat  : PairNat → CNat;
```

for constructing elements of this type from pairs of numbers and for accessing their first and second components.  $\square$

23.4.9 EXERCISE [RECOMMENDED, \*\*\*]: Use the functions defined in Exercise 23.4.8 to write a function `pred` that computes the predecessor of a Church numeral (returning 0 if its input is 0). Hint: the key idea is developed in the example in §5.2. Define a function  $f : \text{PairNat} \rightarrow \text{PairNat}$  that maps the pair  $(i, j)$  into  $(i + 1, i)$ —that is, it throws away the second component of its argument, copies the first component to the second, and increments the first. Then  $n$  applications of  $f$  to the starting pair  $(0, 0)$  yields the pair  $(n, n - 1)$ , from which we extract the predecessor of  $n$  by projecting the second component.  $\square$

23.4.10 EXERCISE [\*\*\*]: There is another way of computing the predecessor function on Church numerals. Let  $k$  stand for the untyped lambda-term  $\lambda x. \lambda y. x$  and  $i$  for  $\lambda x. x$ . The untyped lambda-term

$$\text{vpred} = \lambda n. \lambda s. \lambda z. n (\lambda p. \lambda q. q (p s)) (k z) i$$

(from Barendregt, 1992, who attributes it to J. Velmans) computes the predecessor of an untyped Church numeral. Show that this term can be typed in System F by adding type abstractions and applications as necessary and annotating the bound variables in the untyped term with appropriate types. For extra credit, explain why it works!  $\square$

## Encoding Lists

As a final example, let us extend the Church encodings of numbers to lists. This is a nice demonstration of the expressive power of pure System F, since it shows that all of the programming examples in the subsection above on polymorphic list manipulation can actually be expressed in the pure language. (For convenience, we do use the `fix` construct for defining general recursive functions, but essentially the same constructions can be carried out without it. See Exercises 23.4.11 and 23.4.12.)

We saw in Exercise 5.2.8 that lists can be encoded in the untyped lambda-calculus in a fashion quite similar to the encoding of natural numbers. In effect, a number in unary notation is like a list of dummy elements. Generalizing this idea to elements of any type, we arrive at a Church encoding

for lists, where a list with elements  $x$ ,  $y$ , and  $z$  is represented as a function that, given any function  $f$  and starting value  $v$ , calculates  $f\ x\ (f\ y\ (f\ z\ v))$ . In OCaml terminology, a list is represented as its own `fold_right` function.

The type `List X` of lists with elements of type  $X$  is defined as follows:

```
List X =  $\forall R. (X \rightarrow R \rightarrow R) \rightarrow R \rightarrow R$ ;
```

The `nil` value for this representation of lists easy to write.<sup>1</sup>

```
nil =  $\lambda X. (\lambda R. \lambda c:X \rightarrow R \rightarrow R. \lambda n:R. n)$  as List X;
```

```
► nil :  $\forall X. \text{List } X$ 
```

The `cons` and `isnil` operations are also easy:

```
cons =  $\lambda X. \lambda hd:X. \lambda tl:\text{List } X. (\lambda R. \lambda c:X \rightarrow R \rightarrow R. \lambda n:R. c\ hd\ (tl\ [R]\ c\ n))$  as List X;
```

```
► cons :  $\forall X. X \rightarrow \text{List } X \rightarrow \text{List } X$ 
```

```
isnil =  $\lambda X. \lambda l:\text{List } X. l\ [\text{Bool}]\ (\lambda hd:X. \lambda tl:\text{Bool}. \text{false})\ \text{true}$ ;
```

```
► isnil :  $\forall X. \text{List } X \rightarrow \text{Bool}$ 
```

For the `head` operation, we need to work a little harder. The first difficulty is what to do about head of the empty list. We can address this by recalling that, if we have a general fixed point operator in the language, we can use it to construct an expression of any type. In fact, using type abstraction, we can go further and write a single, uniform function that, given a type  $X$ , yields a function from `Unit` to  $X$  that diverges when applied to `unit`.

```
diverge =  $\lambda X. \lambda _:\text{Unit}. \text{fix } (\lambda x:X. x)$ ;
```

```
► diverge :  $\forall X. \text{Unit} \rightarrow X$ 
```

Now we can use `diverge [X] unit` as the “result” of `head [X] nil`.

```
head =  $\lambda X. \lambda l:\text{List } X. l\ [X]\ (\lambda hd:X. \lambda tl:X. hd)\ (\text{diverge } [X]\ \text{unit})$ ;
```

```
► head :  $\forall X. \text{List } X \rightarrow X$ 
```

Unfortunately, this definition is not yet quite what we want: it will *always* diverge, even when applied to non-empty lists. To get the right behavior, we need to reorganize it a little so that `diverge[X]` is not actually passed its `Unit` argument when it is supplied as an argument to `l`. This is accomplished by removing the `unit` argument and changing the type of the first argument to `l` correspondingly:

---

1. The `as` annotation here helps the typechecker print the type of `nil` in a readable form. As we saw in §11.4, all the typecheckers used in this book perform a simple abbreviation-collapsing step before printing types, but the collapsing function is not smart enough to deal automatically with “parametric abbreviations” like `List`.

```

head =
  λX. λl:List X.
    (l [Unit→X] (λhd:X. λtl:Unit→X. λ_:Unit. hd) (diverge [X]))
    unit;

```

► head :  $\forall X. \text{List } X \rightarrow X$

That is, `l` is applied to a function of type  $X \rightarrow (\text{Unit} \rightarrow X) \rightarrow (\text{Unit} \rightarrow X)$  and a base value of type  $\text{Unit} \rightarrow X$ , and it constructs a function of type  $\text{Unit} \rightarrow X$ . In the case where `l` represents the empty list, this result will be `diverge [X]`; but in the case where `l` represents a non-empty list, the result will be a function that takes `unit` and returns the head element of `l`. The result from `l` is applied to `unit` at the end to get the actual head element (or, if we are unlucky, `diverge`), so that `head` has the type we expect.

For the `tail` function, we use the abbreviation `Pair X Y` (generalizing the `PairNat` type from Exercise 23.4.8) for the Church encoding of pairs with first component of type  $X$  and second component of type  $Y$ :

```

Pair X Y =  $\forall R. (X \rightarrow Y \rightarrow R) \rightarrow R$ ;

```

The operations on pairs are simple generalizations of the operations on the type `PairNat` above:

```

► pair :  $\forall X. \forall Y. X \rightarrow Y \rightarrow \text{Pair } X Y$ 
fst :  $\forall X. \forall Y. \text{Pair } X Y \rightarrow X$ 
snd :  $\forall X. \forall Y. \text{Pair } X Y \rightarrow Y$ 

```

Now the `tail` function can be written like this:

```

tail =
  λX. λl: List X.
    (fst [List X] [List X] (
      l [Pair (List X) (List X)]
        (λhd: X. λtl: Pair (List X) (List X).
          pair [List X] [List X]
            (snd [List X] [List X] tl)
            (cons [X] hd (snd [List X] [List X] tl))))
      (pair [List X] [List X] (nil [X]) (nil [X]))));

```

► tail :  $\forall X. \text{List } X \rightarrow \text{List } X$

- 23.4.11 EXERCISE [★★]: Strictly speaking, the examples in this subsection have not been expressed in pure System F, since we used the `fix` operator to construct a value to be “returned” when `head` is applied to an empty list. Write an alternative version of `head` that takes an extra parameter to be returned (instead of diverging) when the list is empty. □

- 23.4.12 EXERCISE [RECOMMENDED, ★★★]: In pure System F (without `fix`), write a function `insert` of type

$$\forall X. (X \rightarrow X \rightarrow \text{Bool}) \rightarrow \text{List } X \rightarrow X \rightarrow \text{List } X$$

that takes a comparison function, a sorted list, and a new element, and inserts the element into the list at the appropriate point (i.e., after all the elements smaller than it). Next, use `insert` to build a sorting function for lists in pure System F.  $\square$

## 23.5 Basic Properties

The fundamental properties of System F are very similar to those of the simply typed lambda-calculus. In particular, the proofs of type preservation and progress are straightforward extensions of the ones we saw in Chapter 9.

- 23.5.1 THEOREM [PRESERVATION]: If  $\Gamma \vdash t : T$  and  $t \rightarrow t'$ , then  $\Gamma \vdash t' : T$ .  $\square$

*Proof:* EXERCISE [RECOMMENDED, ★★★].  $\square$

- 23.5.2 THEOREM [PROGRESS]: If  $t$  is a closed, well-typed term, then either  $t$  is a value or else there is some  $t'$  with  $t \rightarrow t'$ .  $\square$

*Proof:* EXERCISE [RECOMMENDED, ★★★].  $\square$

System F also shares with  $\lambda_{\perp}$  the property of *normalization*—the fact that the evaluation of every well-typed program terminates.<sup>2</sup> Unlike the type safety theorems above, normalization is quite difficult to prove (indeed, it is somewhat astonishing that it holds at all, considering that we can code things like sorting functions in the pure language, as we did in Exercise 23.4.12, without resorting to `fix`). This proof, based on a generalization of the method presented in Chapter 12, was a major achievement of Girard’s doctoral thesis (1972; also see Girard, Lafont, and Taylor, 1989). Since then, his proof technique has been analyzed and reworked by many others; see Gallier (1990).

- 23.5.3 THEOREM [NORMALIZATION]: Well-typed System F terms are normalizing.  $\square$

---

2. Indeed, presentations of System F with more permissive operational semantics based on full beta-reduction have the *strong normalization* property: every reduction path starting from a well-typed term is guaranteed to terminate.

## 23.6 Erasure, Typability, and Type Reconstruction

As we did for  $\lambda_-$  in §9.5, we can define a *type erasure* function mapping System F terms to untyped lambda-terms by stripping out all their type annotations (including all type abstractions and applications):

$$\begin{aligned} \text{erase}(x) &= x \\ \text{erase}(\lambda x:T_1. t_2) &= \lambda x. \text{erase}(t_2) \\ \text{erase}(t_1 t_2) &= \text{erase}(t_1) \text{erase}(t_2) \\ \text{erase}(\lambda X. t_2) &= \text{erase}(t_2) \\ \text{erase}(t_1 [T_2]) &= \text{erase}(t_1) \end{aligned}$$

A term  $M$  in the untyped lambda-calculus is said to be *typable* in System F if there is some well-typed term  $t$  such that  $\text{erase}(t) = m$ . The *type reconstruction* problem then asks, given an untyped term  $m$ , whether we can find some well-typed term that erases to  $m$ .

Type reconstruction for System F was one of the longest-standing problems in the programming languages literature, remaining open from the early 1970s until it was finally settled (negatively) by Wells in the early 1990s.

23.6.1 THEOREM [WELLS, 1994]: It is undecidable whether, given a closed term  $m$  of the untyped lambda-calculus, there is some well-typed term  $t$  in System F such that  $\text{erase}(t) = m$ .  $\square$

Not only full type reconstruction but also various forms of partial type reconstruction are known to be undecidable for System F. For example, consider the following “partial erasure” function, which leaves intact all typing annotations except the arguments to type applications:

$$\begin{aligned} \text{erase}_p(x) &= x \\ \text{erase}_p(\lambda x:T_1. t_2) &= \lambda x:T_1. \text{erase}_p(t_2) \\ \text{erase}_p(t_1 t_2) &= \text{erase}_p(t_1) \text{erase}_p(t_2) \\ \text{erase}_p(\lambda X. t_2) &= \lambda X. \text{erase}_p(t_2) \\ \text{erase}_p(t_1 [T_2]) &= \text{erase}_p(t_1) [] \end{aligned}$$

Note that type applications are still marked (with empty square brackets) in the erased terms; we can see where they must occur, but not what type must be supplied.

23.6.2 THEOREM [BOEHM 1985, 1989]: It is undecidable whether, given a closed term  $s$  in which type applications are marked but the arguments are omitted, there is some well-typed System F term  $t$  such that  $\text{erase}_p(t) = s$ .  $\square$

Boehm showed that this form of type reconstruction was just as hard as higher-order unification, hence undecidable. Interestingly, this negative result led directly to a useful partial type reconstruction technique (Pfenning,

1988, 1993a) based on Huet's earlier work on efficient semi-algorithms for higher-order unification (Huet, 1975). Later improvements in this line of development have included using a more refined algorithm for higher-order constraint solving (Dowek, Hardin, Kirchner, and Pfenning, 1996), eliminating the troublesome possibilities of nontermination or generation of non-unique solutions. Experience with related algorithms in languages such as LEAP (Pfenning and Lee, 1991), Elf (Pfenning, 1989), and FX (O'Toole and Gifford, 1989) has shown them to be quite well behaved in practice.

A different approach to partial type reconstruction was sparked by Perry's observation that first-class existential types (see Chapter 24) can be integrated with ML's `datatype` mechanism (Perry, 1990); the idea was further developed by Läuffer and Odersky (Läuffer, 1992; Läuffer and Odersky, 1994). In essence, `datatype` constructors and destructors can be regarded as explicit type annotations, marking where values must be injected into and projected from disjoint union types, where recursive types must be folded and unfolded, and (when existentials are added) where packing and unpacking must occur. This idea was extended to include first-class (impredicative) universal quantifiers by Rémy (1994). A more recent proposal by Odersky and Läuffer (1996), further developed by Garrigue and Rémy (1997), conservatively extends ML-style type reconstruction by allowing programmers to explicitly annotate function arguments with types, which may (unlike the annotations that can be inferred automatically) contain embedded universal quantifiers, thus partly bridging the gap between ML and more powerful impredicative systems. This family of approaches to type reconstruction has the advantage of relative simplicity and clean integration with the polymorphism of ML.

A pragmatic approach to partial type reconstruction for systems involving both *subtyping* and impredicative polymorphism, called *local type inference* (or *local type reconstruction*), was proposed by Pierce and Turner (1998; also see Pierce and Turner, 1997; Hosoya and Pierce, 1999). Local type inference has appeared in several recent language designs, including GJ (Bracha, Odersky, Stoutamire, and Wadler, 1998) and Funnel (Odersky and Zenger, 2001), the latter introducing a more powerful form called *colored local type inference* (Odersky, Zenger, and Zenger, 2001).

A simpler but less predictable *greedy type inference* algorithm was proposed by Cardelli (1993); similar algorithms have also been used in proof-checkers for dependent type theories, such as NuPrl (Howe, 1988) and Lego (Pollack, 1990). The idea here is that any type annotation may be omitted by the programmer: a fresh unification variable  $X$  will be generated for each one by the parser. During typechecking, the subtype-checking algorithm may be asked to check whether some type  $S$  is a subtype  $T$ , where both  $S$  and  $T$  may contain unification variables. Subtype-checking proceeds as usual until a

subgoal of the form  $X <: T$  or  $T <: X$  is encountered, at which point  $X$  is instantiated to  $T$ , thus satisfying the immediate constraint in the simplest possible way. However, setting  $X$  to  $T$  may not be the best possible choice, and this may cause later subtype-checks for types involving  $X$  to fail when a different choice would have allowed them to succeed; but, again, practical experience with this algorithm in Cardelli's implementation and in an early version of the Pict language (Pierce and Turner, 2000) shows that the algorithm's greedy choice is correct in nearly all cases. However, when it goes wrong, the greedy algorithm's behavior can be quite puzzling to the programmer, yielding mysterious errors far from the point where a suboptimal instantiation is made.

23.6.3 EXERCISE [\*\*\*]: The normalization property implies that the untyped term  $\text{omega} = (\lambda x. x x) (\lambda y. y y)$  cannot be typed in System F, since reduction of  $\text{omega}$  never reaches a normal form. However, it is possible to give a more direct, “combinatorial” proof of this fact, using just the rules defining the typing relation.

1. Let us call a System F term *exposed* if it is a variable, an abstraction  $\lambda x:T. t$ , or an application  $t s$  (i.e., if it is *not* a type abstraction  $\lambda X. t$  or type application  $t [S]$ ).

Show that if  $t$  is well typed (in some context) and  $\text{erase}(t) = m$ , then there is some exposed term  $s$  such that  $\text{erase}(s) = m$  and  $s$  is well typed (possibly in a different context).

2. Write  $\lambda \bar{X}. t$  as shorthand for a nested sequence of type abstractions of the form  $\lambda X_1 \dots \lambda X_n. t$ . Similarly, write  $t [\bar{A}]$  for a nested sequence of type applications  $((t [A_1]) \dots [A_{n-1}]) [A_n]$  and  $\forall \bar{X}. T$  for a nested sequence of polymorphic types  $\forall X_1 \dots \forall X_n. T$ . Note that these sequences are allowed to be empty. For example, if  $\bar{X}$  is the empty sequence of type variables, then  $\forall \bar{X}. T$  is just  $T$ .

Show that if  $\text{erase}(t) = m$  and  $\Gamma \vdash t : T$ , then there exists some  $s$  of the form  $\lambda \bar{X}. (u [\bar{A}])$ , for some sequence of type variables  $\bar{X}$ , some sequence of types  $\bar{A}$ , and some exposed term  $u$ , with  $\text{erase}(s) = m$  and  $\Gamma \vdash s : T$ .

3. Show that if  $t$  is an exposed term of type  $T$  (under  $\Gamma$ ) and  $\text{erase}(t) = m n$ , then  $t$  has the form  $s u$  for some terms  $s$  and  $u$  such that  $\text{erase}(s) = m$  and  $\text{erase}(u) = n$ , with  $\Gamma \vdash s : U \rightarrow T$  and  $\Gamma \vdash u : U$ .
4. Suppose  $x:T \in \Gamma$ . Show that if  $\Gamma \vdash u : U$  and  $\text{erase}(u) = x x$ , then either
  - (a)  $T = \forall \bar{X}. X_i$ , where  $X_i \in \bar{X}$ , or else
  - (b)  $T = \forall \bar{X}_1 \bar{X}_2. T_1 \rightarrow T_2$ , where  $[\bar{X}_1 \bar{X}_2 \mapsto A]T_1 = [\bar{X}_1 \mapsto B](\forall \bar{Z}. T_1 \rightarrow T_2)$  for some sequences of types  $\bar{A}$  and  $\bar{B}$  with  $|\bar{A}| = |\bar{X}_1 \bar{X}_2|$  and  $|\bar{B}| = |\bar{X}_1|$ .

5. Show that if  $\text{erase}(s) = \lambda x.m$  and  $\Gamma \vdash s : S$ , then  $S$  has the form  $\forall \bar{X}. S_1 \rightarrow S_2$ , for some  $\bar{X}$ ,  $S_1$ , and  $S_2$ .
6. Define the *leftmost leaf* of a type  $T$  as follows:

$$\begin{aligned} \text{leftmost-leaf}(X) &= X \\ \text{leftmost-leaf}(S \rightarrow T) &= \text{leftmost-leaf}(S) \\ \text{leftmost-leaf}(\forall \bar{X}. S) &= \text{leftmost-leaf}(S). \end{aligned}$$

Show that if  $[\bar{X}_1 \bar{X}_2 \mapsto \bar{A}](\forall \bar{Y}. T_1) = [\bar{X}_1 \mapsto \bar{B}](\forall \bar{Z}. (\forall \bar{Y}. T_1) \rightarrow T_2)$ , then it must be the case that  $\text{leftmost-leaf}(T_1) = X_i$  for some  $X_i \in \bar{X}_1 \bar{X}_2$ .

7. Show that  $\omega$  is not typable in System F. □

## 23.7 Erasure and Evaluation Order

The operational semantics given to System F in Figure 23-1 is a *type-passing semantics*: when a polymorphic function meets a type argument, the type is actually substituted into the body of the function. The ML implementation of System F in Chapter 25 does exactly this.

In a more realistic interpreter or compiler for a programming language based on System F, this manipulation of types at run time could impose a significant cost. Moreover, it is easy to see that type annotations play no significant role at run time, in the sense that no run-time decisions are made on the basis of types: we can take a well-typed program, rewrite its type annotations in an arbitrary way, and obtain a program that behaves just the same. For these reasons, many polymorphic languages instead adopt a *type-erasure semantics*, where, after the typechecking phase, all the types are erased and the resulting *untyped* terms are interpreted or compiled to machine code.<sup>3</sup>

However, in a full-blown programming language, which may include side-effecting features such as mutable reference cells or exceptions, the type-erasure function needs to be defined a little more delicately than the full erasure function in §23.6. For example, if we extend System F with an exception-raising primitive `error` (§14.1), then the term

```
let f = (λX.error) in 0;
```

3. In some languages, the presence of features like *casts* (§15.5) forces a type-passing implementation. High-performance implementations of these languages typically attempt to maintain only a vestigial form of type information at run time, e.g., passing types only to polymorphic functions where they may actually be used.

evaluates to 0 because  $\lambda X.\text{error}$  is a syntactic value and the `error` in its body is never evaluated, while its erasure

```
let f = error in 0;
```

raises an exception when evaluated.<sup>4</sup> What this shows is that type abstractions *do* play a significant semantic role, since they stop evaluation under a call-by-value evaluation strategy and hence can postpone or prevent the evaluation of side-effecting primitives.

We can repair this discrepancy by introducing a new form of erasure appropriate for call-by-value evaluation, in which we erase a type abstraction to a *term*-abstraction

$$\begin{aligned} \text{erase}_v(x) &= x \\ \text{erase}_v(\lambda x:T_1. t_2) &= \lambda x. \text{erase}_v(t_2) \\ \text{erase}_v(t_1 t_2) &= \text{erase}_v(t_1) \text{erase}_v(t_2) \\ \text{erase}_v(\lambda X. t_2) &= \lambda_. \text{erase}_v(t_2) \\ \text{erase}_v(t_1 [T_2]) &= \text{erase}_v(t_1) \text{dummy}_v \end{aligned}$$

where `dummyv` is some arbitrary untyped value, such as `unit`.<sup>5</sup> The appropriateness of this new erasure function is expressed by the observation that it “commutes” with untyped evaluation, in the sense that erasure and evaluation can be performed in either order:

23.7.2 THEOREM: If  $\text{erase}_v(t) = u$ , then either (1) both  $t$  and  $u$  are normal forms according to their respective evaluation relations, or (2)  $t \rightarrow t'$  and  $u \rightarrow u'$ , with  $\text{erase}_v(t') = u'$ .  $\square$

## 23.8 Fragments of System F

The elegance and power of System F have earned it a central role in theoretical studies of polymorphism. For language design, however, the loss of type

---

4. This is related to the problem we saw with the unsound combination of references and ML-style `let`-polymorphism in §22.7. The *generalization* of the `let`-body in that example corresponds to the explicit type abstraction here.

23.7.1 EXERCISE [★★]: Translate the unsound example on page 335 into System F extended with references (Figure 13-1).  $\square$

5. In contrast, the *value restriction* that we imposed in order to recover soundness of ML-style type reconstruction in the presence of side effects in §22.7 does erase type-abstractions—generalizing a type variable is essentially the opposite of erasing a type abstraction—but ensures soundness by permitting such generalizations only when the inferred type abstraction would occur immediately adjacent to a term abstraction or other syntactic value-constructor, since these also stop evaluation.

reconstruction is sometimes considered to be too heavy a price to pay for a feature whose full power is seldom used. This has led to various proposals for restricted fragments of System F with more tractable reconstruction problems.

The most popular of these is the let-polymorphism of ML (§22.7), which is sometimes called *prenex polymorphism* because it can be viewed as a fragment of System F in which type variables range only over quantifier-free types (*monotypes*) and in which quantified types (*polytypes*, or *type schemes*) are not allowed to appear on the left-hand sides of arrows. The special role of `let` in ML makes the correspondence slightly tricky to state precisely; see Jim (1995) for details.

Another well-studied restriction of System F is *rank-2 polymorphism*, introduced by Leivant (1983) and further investigated by many others (see Jim, 1995, 1996). A type is said to be of rank 2 if no path from its root to a  $\forall$  quantifier passes to the left of 2 or more arrows, when the type is drawn as a tree. For example,  $(\forall X. X \rightarrow X) \rightarrow \text{Nat}$  is of rank 2, as are  $\text{Nat} \rightarrow \text{Nat}$  and  $\text{Nat} \rightarrow (\forall X. X \rightarrow X) \rightarrow \text{Nat} \rightarrow \text{Nat}$ , but  $((\forall X. X \rightarrow X) \rightarrow \text{Nat}) \rightarrow \text{Nat}$  is not. In the rank-2 system, all types are restricted to be of rank 2. This system is slightly more powerful than the prenex (ML) fragment, in the sense that it can assign types to more untyped lambda-terms.

Kfoury and Tiuryn (1990) proved that the complexity of type reconstruction for the rank-2 fragment of System F is identical to that of ML (i.e., DEXPTIME-complete). Kfoury and Wells (1999) gave the first correct type reconstruction algorithm for the rank 2 system and showed that type reconstruction for ranks 3 and higher of System F is undecidable.

The rank-2 restriction can be applied to other powerful type constructors besides quantifiers. For example, *intersection types* (see §15.7) can be restricted to rank 2 by excluding types in which an intersection appears to the left of 2 or more arrows (Kfoury, Mairson, Turbak, and Wells, 1999). The rank-2 fragments of System F and of the first-order intersection type system are closely related. Indeed, Jim (1995) showed that they can type exactly the same untyped terms.

## 23.9 Parametricity

Recall from §23.4 how we defined the type `CBool` of Church booleans

$$\text{CBool} = \forall X. X \rightarrow X \rightarrow X;$$

and the constants `tru` and `f1s`:

$$\text{tru} = \lambda X. \lambda t:X. \lambda f:X. t;$$

```

► tru : CBool
  fls = λX. λt:X. λf:X. f;
► fls : CBool

```

Given the type `CBool`, we can actually write the definitions of `tru` and `fls` rather mechanically, simply by looking at the structure of the type. Since `CBool` begins with a  $\forall$ , any value of type `CBool` must be a type abstraction, so `tru` and `fls` must both begin with a  $\lambda X$ . Then, since the body of `CBool` is an arrow type  $X \rightarrow X \rightarrow X$ , every value of this type must take two arguments of type  $X$ —i.e., the bodies of `tru` and `fls` must each begin  $\lambda t:X. \lambda f:X$ . Finally, since the result type of `CBool` is  $X$ , any value of type `CBool` must return an element of type  $X$ . But since  $X$  is a parameter, the only values of this type that we can possibly return are the bound variables `t` and `f`—we have no other way of obtaining or constructing values of this type ourselves. In other words, `tru` and `fls` are essentially the *only* inhabitants of the type `CBool`. Strictly speaking, `CBool` contains some other terms like  $(\lambda b:CBool. b) \text{tru}$ , but it is intuitively clear that every one of them must behave like either `tru` or `fls`.

This observation is a simple consequence of a powerful principle known as *parametricity*, which formalizes the uniform behavior of polymorphic programs. Parametricity was introduced by Reynolds (1974, 1983) and has been further explored, along with related notions, by Reynolds (1984, Reynolds and Plotkin, 1993), Bainbridge et al. (1990), Ma (1992), Mitchell (1986), Mitchell and Meyer (1985), Hasegawa (1991), Pitts (1987, 1989, 2000), Abadi, Cardelli, Curien, and Plotkin (Abadi, Cardelli, and Curien, 1993; Plotkin and Abadi, 1993; Plotkin, Abadi, and Cardelli, 1994), Wadler (1989, 2001), and others. See Wadler (1989) for an expository introduction.

### 23.10 Impredicativity

The polymorphism of System F is often called *impredicative*. In general, a definition (of a set, a type, etc.) is called “impredicative” if it involves a quantifier whose domain includes the very thing being defined. For example, in System F, the type variable  $X$  in the type  $T = \forall X. X \rightarrow X$  ranges over all types, including  $T$  itself (so that, for example, we can instantiate a term of type  $T$  at type  $T$ , yielding a function from  $T$  to  $T$ ). The polymorphism found in ML, on the other hand, is often called *predicative* (or *stratified*), because the range of type variables is restricted to monotypes, which do not contain quantifiers.

The terms “predicative” and “impredicative” originate in logic. Quine (1987) offers a lucid summary of their history:

In exchanges with Henri Poincaré . . . Russell attributed [Russell's] paradox tentatively to what he called a vicious-circle fallacy. The “fallacy” consisted in specifying a class by a membership condition that makes reference directly or indirectly to a range of classes one of which is the very class that is being specified. For instance the membership condition behind Russell's Paradox is non-self-membership:  $x$  not a member of  $x$ . The paradox comes of letting the  $x$  of the membership condition be, among other things, the very class that is being defined by the membership condition. Russell and Poincaré came to call such a membership condition *impredicative*, and disqualified it as a means of specifying a class. The paradoxes of set theory, Russell's and others, were thus dismantled. . .

Speaking of terminology, whence “predicative” and “impredicative”? Our tattered platitude about classes and membership conditions was, in Russell's phrase, that every predicate determines a class; and then he accommodates the tattering of the platitude by withdrawing the title of predicate from such membership conditions as were no longer to be seen as determining classes. “Predicative” thus did not connote the hierarchical approach in particular, or the metaphor of progressive construction; that was just Russell and Poincaré's particular proposal of what membership conditions to accept as class-productive, or “predicative.” But the tail soon came to wag the dog. Today predicative set theory is constructive set theory, and impredicative definition is strictly as explained in the foregoing paragraph, regardless of what membership conditions one may choose to regard as determining classes.

## 23.11 Notes

Further reading on System F can be found in Reynolds's introductory article (1990) and his *Theories of Programming Languages* (1998b).